Algebraic Reasoning Distiller: Harnessing multi-agent collaboration for enhanced mathematical problem-solving

Joaquín Mir Macías
joaquinmirmacias@icai.alu.comillas.es

Jose Andrés Ridruejo Tuñón
202111246@alu.comillas.edu

Javier Rios Montes
202111258@alu.comillas.edu

Miguel Montes Lorenzo
miguelmonteslorenzo@alu.comillas.edu

Abstract

This project explores the potential of multi-agent collaboration in enhancing the mathematical problem-solving capabilities of large language models (LLMs). We propose a pipeline that integrates six different agents, each specializing in a distinct aspect of mathematical reasoning, to collaboratively solve complex problems. The agents include a Planner, a Counterexample Generator, a Symbolic Prover, a RAG Retriever, an Aggregator, and a Distiller. Our results show that training and running this agent *offline* can be a powerful way to distill mathematical knowledge into a compact file, which can then be used to solve problems in an *online* setting with a single call to a constrained LLM. This enables any base model to solve complex mathematical problems effectively without the need to perform expensive reasoning during inference.

1 Introduction

In the last few years, large language models (**LLMs**) have positioned themselves as powerful tools for a wide range of natural language processing tasks. However, their performance in *mathematical problem-solving* has been limited by their inability to perform complex reasoning and to handle symbolic manipulation effectively. Unlike human mathematicians, who are able to explicitly take into account symbolic rules and logical deductions, LLMs often operate as parrots (though stochastic ones, we must admit), aproximating solutions based on patterns rather than understanding the underlying mathematical principles. This shows in consistent errors in even simple arithmetic, spatial reasoning and symbolic manipulation tasks. Even frontier models like GPT-5 have shown these limitations, which suggests memorized reasoning templates rather than abstract understanding.

To overcome these issues, two main approaches have been explored in the literature: **pretraining** and **fine-tuning**. Pretraining involves training a model on a large corpus of mathematical texts, allowing it to learn general mathematical concepts and patterns. However, this approach can be computationally expensive. On the other hand, given both a pretrained model and a curated dataset of mathematical problems, fine-tuning allows the model to adapt to specific mathematical tasks, improving its performance without the need for extensive computational resources. This latter approach initially implied trading off generalization for specialization, turning the model into a *narrow expert*.

Recent work, though, introduced a novel paradigm that combines the strengths of both approaches: **Supervised Fine-tuning**. First theorically proposed by Stiennon et al. [1] and further implemented with **InstructGPT** [2], this method involves using a curated dataset of outputs (examples of an LLM behaving as desired) to fine-tune a pretrained model. Given that this process just tunes the

model to emulate a correct style, rathen than to solve a particular task, it does not lose its generic problem-solving capabilities. In fact, it can even enhance them by providing the model with a better understanding of how to approach problems in a structured way.

Our project builds upon this idea of supervised fine-tuning, but takes it a step further by introducing a multi-agent collaboration framework. Instead of relying on a single model to solve mathematical problems, we propose a system where multiple specialized agents work together to tackle different aspects of the problem-solving process. Each agent is designed to handle specific tasks. By leveraging the strengths of each agent and allowing them to collaborate, we aim to create a more robust and effective system for mathematical problem-solving. In particular, we will apply this framework to the **Mathematics Distillation Challenge - Equational Theories** [3], which presents a set of complex mathematical problems that require a deep understanding of equational theories.

1.1 Problem Statement

Let a **magma** be a set M equipped with a binary operation $\cdot : M \times M \to M$ (no associativity, commutativity or identity element are assumed). An **equational theory** is a set of equations that hold for all elements of the magma. The goal of the Mathematics Distillation Challenge - Equational Theories is to develop a system that can effectively solve problems related to equational theories. Specifically, given two equations E_1 and E_2, the system should determine whether E_1 implies E_2 under every interpretation of the magma (this is, if every magma satisfying E_1 also satisfies E_2). A given example of this problem is the following:

$$\begin{aligned} x * y &= z * (w * (u * u)) \\ x * (y * y) &= (z * w) * z \end{aligned} \tag{1}$$

In this case, the equation is **True**. The system must generate a cheatsheet that contains the necessary information to solve this kind of problems (which is then fed to a constrained LLM that must solve the problem based on the information in the cheatsheet). The cheatsheet should include relevant definitions, properties, and examples of equational theories, as well as any other information that can help the LLM to understand and solve the problem effectively.

The competition operates under a set of constraints that limit the computational resources available to the participants:

- The generated cheatsheet must be static and no greater than 10 KB in size.
- No tools may be used during the evaluation phase, and the LLM must rely solely on the information provided in the cheatsheet to solve the problem.
- The budget for each problem is limited to 10 minutes and the equivalent of 0.01 USD in computational resources.

These constraints motivate our approach of *offline* distillation: the computationally expensive parts of the process (counterexample search, RAG,multi-agent reasoning, etc.) are performed during the training phase. during the evaluation phase, the system only needs the use of a cheatsheet and a single call to the LLM.

2 Method

Our solution, ARD (Algebraic Reasoning Distiller), follows the paradigm **distill, then deploy**. In its *offline* setting, the multi-agent system analyzes the training dataset of problems and‘produces *evidence bundles*. These are then used to both generate the final cheatsheet and to fine-tune a pretrained LLM, which is then deployed in the *online* setting to solve the test problems based on the generated cheatsheet.

2.1 System Architecture

The multi-agent system consists of 5 sequential nodes, each responsible for a specific aspect of the problem-solving process. Below, we include a graphic to illustrate the architecture of the system, followed by a detailed description of each agent's role and functionality:

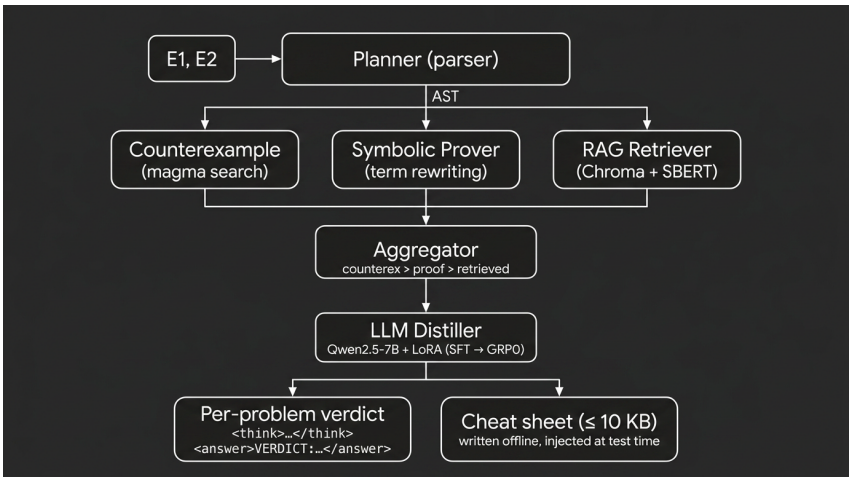

Figure 1: Architecture of the multi-agent system.

2.1.1 Planner

This agent takes the input problem ((E_1, E_2)) and generates parses them into **AST** (Abstract Syntax Tree) format. Terms are represented as nested tuples: `("var", "x")` for variables and `("op", left, right)` for operations. It implements a simple recursive descent parser to achieve this.

2.1.2 RAG Retriever

Recovers relevant information from a knowledge base of equational theories. In specific, it uses a vector database (ChromaDB) which contains the 4,694 equations from the training set, embedded using a pretrained model (`all-MiniLM-L6-v2`). The agent retrieves the *top-k* (in our case, $k = 5$) most relevant equations based on cosine similarity.

2.1.3 Symbolic Prover

This agent attempts to prove the implication $E_1 \implies E_2$ using symbolic manipulation. It applies a predefined list of strategies:

1. **Alpha-equivalence**: Check if E_1 and E_2 are the same up to variable renaming.
2. **Substitution**: Attempt to substitute terms in E_1 to derive E_2.
3. **Rewriting**: Use the retrieved equations to rewrite E_1 and see if it can be transformed into E_2.

This component is deliberately designed to be overly conservative: a negative result does not necessarily mean that the implication is false, but a positive result is a strong signal of the implication being true.

2.1.4 Counterexample Generator

Tries to find a counterexample that satisfies E_1 but not E_2. It searches increasingly complex magmas (starting from a single-element set and increasing the number of elements) and checks if there exists an interpretation that satisfies E_1 but not E_2, applying a different heuristic depending on the order:

- **Orders 2 and 3**: Exhaustive search over all possible interpretations. There are only 16 possible magmas of order 2 and 19,683 of order 3, which makes this approach feasible.
- **Order 4:** Random sampling of 20,000 Cayley tables [1] (out of the over 4 billion possible magmas of order 4) and checking if any of them satisfies E_1 but not E_2.

[1] A Cayley table is a mathematical table that describes the operation of a magma. It lists the results of applying the binary operation to every possible pair of elements in the set.

For every candidate, all possible variable assignments are checked by means of a cartesian product iteration with mixed-radix counting (to account for the different number of elements in each order). A single counterexample is sufficient to conclude that the implication is false, but if no counterexample is found, no conclusion can be drawn.

2.1.5 Aggregator

The aggregator takes the outputs of the previous agents and generates a priority policy:

- If the Symbolic Prover finds a proof, the implication is classified as **True** with confidence 0.9.
- If the Counterexample Generator finds a counterexample, the implication is classified as **False** with confidence 1.0.
- If neither of the above agents finds a conclusive result, the implication is classified as None with confidence 0.0.

2.2 Architectural Details

In addition to the specific algorithms and heuristics implemented in each agent, there are two key architectural details that are worth mentioning:

- **Distilled LLM Agent**: This agent is responsible for generating the final cheatsheet based on the evidence bundles produced by the multi-agent system. It uses a pretrained LLM, in our case Qwen2.5-7B-Instruct [4], quantized to 4 bits and using BF16 precision for the attention mechanism. The model is fine-tuned on the training dataset using LoRA [5] with a rank of 16, α of 32 and a dropout of 0.05. During inference, the model operates under two different modes:
  - **Problem-specific:** Given an evidence bundle and the (optional) retrieved context, the model generates an structured answer that includes the tokens `<think>`, `</think>`, `<answer>` and `</answer>`.
  - **General-purpose:** Given a set of evidence bundles, the model generates a compact *lemma/heuristic* block using greedy decoding. This block is designed to be as informative as possible while being no greater than 1200 bytes in size (in order to fit the 10 KB limit for the final cheatsheet after including the retrieved context).
- **FastAPI Server**: A lightweight API REST exposes the functionality of the multi-agent system, allowing the access through two endpoints: POST `/sair` (which runs the complete system under a given problem, returning a `SAIRResponse` object) and GET `/health` (checking that the server is running properly).

3 Experiments and Results

All the training and evaluation processes were conducted on a node of the DGX cluster (8 $\times$ NVIDIA H100 GPUs). The training dataset consists of 4,694 problems [2], divided into a training set consisting of an 80% of the problems (3,755) and a validation set consisting of the remaining 20% (939). Despite this, not all problems were used for fine-tuning the Distilled LLM Agent, as we only used those for which the multi-agent system was able to produce a coherent verdict with that of the ground truth. This decision was made to ensure that the model learns from correct examples and to avoid introducing noise into the training process. The final corpus consisted of around 1330 problems.

Our experiments go along two main lines:

- **Evaluation of the complete system:** We evaluate the performance of the complete system (this is, the fine-tuned Distilled LLM Agent with the complete cheatsheet) on a test set, which consists of 1,000 problems of normal difficulty.
- **Ablation study on the cheatsheet:** We conduct an ablation study to analyze the contribution of different components of the generated cheatsheet to the performance of the system. In this

[2] These were obtained from various sources, mainly the competition dataset

case, we test the performance of a base model (without fine-tuning) with different variants of the generated cheatsheet, which are derived from the original one by removing specific components (such as symbolic heuristics or countermodel library).

3.1 Evaluation Metrics

The performance of the system was evaluated mainly through the accuracy in the classification. Added to this, we include other auxiliary metrics such as any bias towards one of the classes (True or False). More advanced metrics (such as those pertaining the quality of the generated reasoning or the interpretability of the generated cheatsheet) were not considered in this work due to time constraints, but they are an interesting line of future research.

3.2 Main Results

The complete system achieved an accuracy of **0.765** on the test set. Despite these promising results, the system still shows some limitations, such as a bias towards classifying problems as **False**, as we can see in Figure 2.

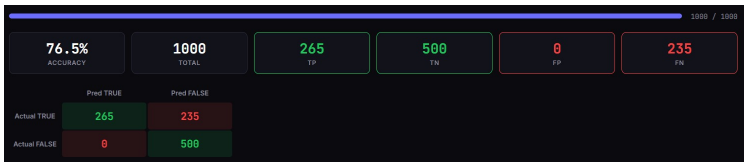

Figure 2: Confusion matrix of the complete system on the test set.

This partiality may be due to two main factors: (i) the generated cheatsheet may be providing too many patterns and heuristics for finding counterexamples, which may serve as a sort of *inductive bias* towards classifying problems as **False** (as finding a single counterexample is sufficient to classify a problem as **False**, while proving that an implication holds in all magmas is a much more complex task), and (ii) LLMs might, by pure nature, assume that the absence of a proof is a strong signal of the implication being false, which is not necessarily the case in this kind of problems (as we have seen, the symbolic prover is designed to be conservative, so a negative result does not necessarily mean that the implication is false). These limitations are intriguing and show up again in the following section.

3.3 Cheatsheet Ablation Study

3.3.1 Methodology

Instead of using our fine-tuned Distilled LLM Agent to generate the final verdict, we conducted an ablation study over a base model provided by the competition organizers (`OPENAI: gpt-oss-120b`), which is a large language model pretrained on a wide range of tasks but not fine-tuned for our specific problem. This choice allows us to isolate the impact of the generated cheatsheet on the performance of the system, as any improvement in the metrics can be attributed to the quality and relevance of the information included in the cheatsheet (instead of having to deal with the disentanglement of the improvements due to the fine-tuning of the model and the improvements due to the generated content in the cheatsheet). In this way, we can better understand the contribution of our multi-agent system in generating useful information for solving the problems, regardless of the specific capabilities of the LLM used for inference.

This has one main drawback, though: Due to the constraints in number of calls (set to 100 problems per account) and computational resources, we only had the chance to run a limited number of test problems for each configuration, which may have an impact on the statistical significance of the results. A sword has two edges, though, and this limitation is also an advantage in the sense that it allows us to compare the results from previous experiments (on the fine-tuned model) with the results from this ablation study, which enables us to have a more comprehensive understanding of the performance of the system and the contribution of each component (the fine-tuning of the model and the generated content in the cheatsheet) to the final results.

In order to keep the comparations as fair as possible, we tested the same set of 50 problems for each configuration, which were randomly sampled from the test set provided by the competition organizers. This set was chosen so that 30 problems were of *normal* difficulty and 20 were of *hard* difficulty, in order to have a representative sample of the different types of problems that the system may encounter during the evaluation phase. Additionally, to avoid any potential bias in the results, half of the problems were sampled from the subset with ground truth **True** and the other half from the subset with ground truth **False**. The complete list of the sampled problems is included in Appendix B.1.

We try five difference variants derived from the original cheatsheet, which are the following:

- **Full Cheatsheet**: The complete cheatsheet generated by the Distilled LLM Agent, including both the problem-specific answer and the general-purpose lemma/heuristic block (as it is quite extense, we don't include it in the report, but it is available in the code repository, under `sair/cheatsheets/final_cheatsheet.txt`).
- **No Symbolic TRUE heuristics**: We remove the part dubbed as *Symbolic / Rewriting intuition* in the original cheatsheet, which includes heuristics and strategies such as *syntactic identity up to variable renaming*, *substitution* and *projection-based reasoning*.
- **No Countermodel Library**: We remove that component which is mainly responsible for quick search of counterexamples and canonical countermodels.
- **Neither of the above**: We remove both the symbolic heuristics and the countermodel library, leaving only global instructions, output formatting, generic proof strategy and lemma pack stubs.
- **Baseline**: A basic cheatsheet that only includes what the LLM must do (with no specific information about the how) and the expected structure of the output (required to be in the cheatsheet by the competition organizers). This configuration serves as a baseline to compare the performance of the other configurations and to assess the contribution of the generated content in the cheatsheet. We include this cheatsheet in Appendix B.2.

Specific information on the parts removed in the variants **No Symbolic TRUE heuristics** and **No Countermodel Library** can be found in Appendices B.3 and B.4, respectively.

3.3.2 Results

We have included, in the Appendix A, an example of execution on the baseline cheatsheet and the full cheatsheet. We recommend the reader to consult these examples for a better understanding of the system's behavior, but they suggest that the complete cheat sheet is not just adding more text: it provides a better search space for reasoning. It would be interesting to analyze the interpretability of the generated cheatsheets and the outputs to have a deeper understanding of the reasoning process of the system beyond the evaluation metrics.

In Table 1 we show the results obtained for each configuration, where `TF`, f.e., stands for the problems that were classified as **True** by the model but are actually **False** according to the ground truth (this is, code `<predicted_label><ground_truth>`).

Variant	TT	TF	FT	FF	NV	Acc.	Err.
Full Cheatsheet	24	10	1	14	1	0.76	0.22
No Symbolic True	22	9	2	17	0	0.78	0.22
No Countermodel	24	10	0	14	2	0.76	0.20
Neither	21	15	4	10	0	0.62	0.38
Baseline	16	11	9	14	0	0.60	0.40

Table 1: Results of the ablation study on the cheatsheet.

Some interesting insights from these results are the following:

- Removing the symbolic heuristics seems to have a positive impact on the performance of the system, which may simply be an statistical fluctuation due to the small sample size, but

it may also be a consequence of the fact that some of the heuristics included in the generated cheatsheet are not actually helpful for solving the problems.

- While removing the individual components (symbolic heuristics and countermodel library) does not seem to have a significant impact on the performance of the system, removing both of them at the same time has a significant negative impact on the performance. This is quite a confusing result, as it may suggest that there is some sort of synergy between the two components.
- The baseline configuration shows a significant drop in performance, which suggests that the generated content in the cheatsheet is indeed contributing to the performance of the system. However, it is worth noting that the baseline configuration still achieves an accuracy of 0.60, which is not negligible and may be due to the fact that the base model (gpt-oss-120b) has some inherent capabilities for solving this kind of problems, even without the fine-tuning and the generated content in the cheatsheet. This seems reasonable, as it was released in August 2025 and is a large language model pretrained on a wide range of tasks, which may include some mathematical problem-solving capabilities.
- Similar to the results obtained with the complete system, all the configurations show a bias towards classifying problems as **False**.

All in all, we can conclude that the inclusion of specific heuristics and strategies for solving the problems has a positive impact on the performance of the system (increasing the accuracy by a quarter in comparison to the baseline configuration), but there is still room for improvement, as the system still shows some limitations and biases.

4 Conclusion

In this project, we have explored the potential of multi-agent collaboration for enhancing the mathematical problem-solving capabilities of large language models. We proposed a pipeline that integrates six different agents, each specializing in a distinct aspect of mathematical reasoning, to collaboratively solve complex problems related to equational theories. Our results have 4 main implications:

- The multi-agent system is effective in generating useful information for solving the problems, as evidenced by the improvement in performance when using the generated cheatsheet compared to a baseline configuration.
- This allows for a *cheaper* (in the computational sense) and more efficient way to adjust our models to specific tasks, rather than having to perform expensive reasoning during inference. This advantage is relevant as the greatest part of the processing may be performed offline, which allows us to leverage more powerful models and more computational resources during the training phase, while still being able to deploy the system with a constrained LLM during the evaluation phase.
- LLMs may have some inherent bias that leads them to take actions which may not be the most optimal for solving the problems, as well as taking logical leaps which may not be justified (such as assuming that the absence of a proof is a strong signal of the implication being false). This is an interesting insight that may be worth exploring in future research, as it may have implications for the design of prompts and the development of more effective strategies for guiding the reasoning process of LLMs.
- The generated cheatsheet is not just adding more text; it is providing a better search space for reasoning, which allows the model to have a more structured and informed approach to solving the problems.

References

[1] Nisan Stiennon, Long Ouyang, Jeff Wu, Daniel Krueger, Milan Goel, Joshua Zhen, Dhruba Ganguli, Sandhini Agarwal, Danny Mislra, and Pieter Christiano. Learning to summarize from human feedback. *Advances in Neural Information Processing Systems*, 33:3008–3021, 2020.

[2] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Agarwal, Chen Wu, Danny Hadfield-Menell, Pieter Christiano, and Timothy Lyons. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022. URL `https://arxiv.org/abs/2203.02155`.

[3] SAIR Foundation. Equational theories stage 1: Mathematics distillation challenge. `https://github.com/sair-foundation/equational-theories-stage1-judge`, 2026. Accessed: 2026-04-19.

[4] Qwen Team. Qwen2.5-7b-instruct model card. `https://huggingface.co/Qwen/Qwen2.5-7B`, 2024. Accessed: 2026-04-19.

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. URL `https://arxiv.org/abs/2106.09685`.

A Input/Output Examples

In the following sections, we include some examples of the inputs and outputs of the system for a given problem (Hard `problem_id` 26), which was correctly classified by the complete system as **True** but misclassified as **False** when using the baseline cheatsheet.

The key difference is that the *full cheat sheet* gives the model a correct high-level theory of the problem, while the baseline leaves it with a much thinner and more fragile set of cues. In the full version, the model is explicitly told to search for two complementary routes: a syntactic or rewriting-based path to prove implication, and a finite-countermodel path to refute it. That extra structure matters because the task is not about guessing the most likely answer; it is about deciding whether one equation entails another *in all magmas*, so the model needs a reliable way to distinguish universal validity from a single accidental match.

By contrast, the baseline seems to have missed the crucial left-projection insight and instead gravitated toward a counterexample-style template. In your specific pair, the correct reasoning is that Eq. 1 forces the operation into a very restrictive form, and once that form is recognized, Eq. 2 becomes automatic. The baseline apparently failed to isolate that algebraic consequence and treated the task as if it only needed to find a refuting magma. That is exactly the kind of failure a weak prompt can induce: the model locks onto a familiar "FALSE by counterexample" pattern even when the universal condition is actually strong enough to imply TRUE.

What this suggests about the complete cheat sheet is that it is not just adding more text; it is providing a better search space for reasoning. The model gets explicit pathways for proving true implications, not just for finding countermodels, so it is less likely to default to a pessimistic heuristic. In other words, the full cheat sheet seems to improve performance by balancing the model's prior toward both proof and refutation, whereas the baseline appears to bias it toward the easier and more common failure mode: rejecting statements whenever the structure is not immediately obvious.

A.1 Full Cheatsheet Execution

The prompt, in this case, is the complete cheatsheet generated by the Distilled LLM Agent, with the fields `equation1` and `equation2` filled with the corresponding equations from the problem. We include the generated output from the LLM, which includes the reasoning process and the final verdict:

```
    <think>
We need to decide whether every magma satisfying

  x = x * (y * (x * z))        (Eq 1)

also satisfies

  x = ((x * y) * (z * w)) * u  (Eq 2).

From Eq 1 we can deduce a strong law.

Fix arbitrary elements a, b of the magma.
Take x := a, y := b, z := a. Then Eq 1 gives

  a = a * (b * (a * a)).

Let c := b * (a * a). The equation reads a = a * c.
Since b and a were arbitrary, the element c can be any element
that can be written as b * (a * a). Varying b we see that for the
fixed a we have a = a * d for all elements d of the magma.
Thus for every a and every d,

  a * d = a.

Hence the binary operation is a left projection: the result of
```

```
a * d is always the left argument a. Conversely, any left-projection
magma clearly satisfies Eq 1, so Eq 1 is equivalent to the
left-projection law.

Now evaluate Eq 2 in a left-projection magma. For arbitrary x, y, z, w, u,

  (x * y) = x,     (z * w) = z,

so

  (x * y) * (z * w) = x * z = x,

and then

  ((x * y) * (z * w)) * u = x * u = x.

Thus the right-hand side of Eq 2 reduces to x, and Eq 2 becomes the
trivial identity x = x. Therefore every magma satisfying Eq 1 also
satisfies Eq 2. So the implication holds in all magmas.
</think>

<answer>
VERDICT: TRUE

REASONING: Eq 1 forces the operation to be a left projection
 (a * b = a for all a, b).
In any left-projection magma the right-hand side
 of Eq 2 reduces to x, making Eq 2 an identity.

PROOF: From Eq 1 set x := a, y := b, z := a to obtain a = a * (b * (a * a)).
Let c = b * (a * a). Then a = a * c. Varying b shows that for the fixed a,
a * d = a for every element d; hence a * d = a for all a, d, i.e. the operation
is left projection. In such a magma, (x * y) = x and (z * w) = z, so
((x * y) * (z * w)) * u = (x * z) * u = x * u = x. Thus Eq 2 holds universally.
</answer>
```

A.2 Baseline Cheatsheet Execution

Analogously, the prompt to the LLM in this case is the baseline cheatsheet included in Appendix B.2, with the fields `equation1` and `equation2` filled with the corresponding equations from the problem. The output generated is the following:

```
    <think>
We need a magma where the identity
\[
x = x * (y * (x * z))
\]
holds for all elements, but the identity
\[
x = ((x * y) * (z * w)) * u
\]
fails for some choice of variables.

Take the two-element set \(M = \{0,1\}\) with
 the binary operation defined by
\[
a * b =
\begin{cases}
a & \text{if } a = 0,\\
```

```
b & \text{if } a = 1.
\end{cases}
\]

The table is
\[
\begin{array}{c|cc}
* & 0 & 1\\
\hline
0 & 0 & 0\\
1 & 0 & 1
\end{array}
\]

One checks that for every \(x,y,z \in M\),
\[
x * (y * (x * z)) = x,
\]
so Equation~1 is satisfied universally.

However, choosing \(x = 1\), \(y = 0\),
\(z = 0\), \(w = 0\), \(u = 0\) gives
\[
((x * y) * (z * w)) * u
= ((1 * 0) * (0 * 0)) * 0
= (0 * 0) * 0
= 0 \neq 1 = x.
\]
So Equation~2 fails. Hence Equation~1 does
not imply Equation~2 in all magmas.
</think>

<answer>
\textbf{VERDICT: FALSE}

\textbf{REASONING:} A magma can satisfy the first
 identity for all elements while the second identity
 fails for some substitution, so the implication
 does not hold universally.

\textbf{COUNTEREXAMPLE:} Let \(M = \{0,1\}\) with operation
  \(a*b = a\) if \(a=0\) and \(a*b=b\) if \(a=1\).
   Then \(x = x * (y * (x * z))\) holds
    for all \(x,y,z \in M\), but for \(x = 1\),
    \(y = 0\), \(z = 0\), \(w = 0\), \(u = 0\),
\[
((x * y) * (z * w)) * u = 0 \neq 1 = x.
\]
Thus Equation~2 is not forced by Equation~1.
\end{answer}
```

B Reproducibility

B.1 Problem Execution

We used the cheatsheet and its variants to *help* the LLM solve 50 problems. As mentioned in the main text, these were chosen from the test set provided by the competition organizers, with a representative distribution of difficulties (30 normal and 20 hard) and ground truth labels (25 True and 25 False). In order to provide reproducible results, the given problems were, as listed in the SAIR Playground, the following:

- **Normal:** `problem_ids` 0-19, 20-27, 30 and 31.
- **Hard:** `problem_ids` 0-2, 6, 7, 12, 22, 23, 26, 27, 33-36, 39-44.

B.2 Baseline Cheatsheet

The baseline cheatsheet provided to the LLM in the corresponding configuration of the ablation study is that of the original one, but stripped of all the problem-specific information and general-purpose lemmas/heuristics. It is as follows:

```
## MAGMA IMPLICATION CHEAT SHEET
You are deciding whether Equation 1 implies Equation 2 over all magmas.
A magma is a non-empty set with one binary operation * and no axioms.

OUTPUT:
- VERDICT: TRUE or FALSE
- REASONING: brief but explicit
- PROOF: required for TRUE
- COUNTEREXAMPLE: required for FALSE

## Problem

Equation 1: {{ equation1 }}
Equation 2: {{ equation2 }}

## Required output format

Respond with exactly:

<think>
Your reasoning here.
</think>
<answer>
VERDICT: TRUE
REASONING: ...
PROOF: ...
</answer>

Or if false:

<answer>
VERDICT: FALSE
REASONING: ...
COUNTEREXAMPLE: ...
</answer>
```

B.3 No symbolic TRUE heuristics

The piece removed is the following:

```
    FAST TRUE CHECKS:
```

```
- Eq2 is syntactically identical on both sides, or Eq2 is Eq1 up
  to side swap and variable renaming.
- Eq1 rewrites Eq2's left side to its right side
  by direct substitution instances.
- Eq1 forces a projection law such as a*b=b or a*b=a,
  and Eq2 simplifies under that law.
```

B.4 No countermodel library

The piece removed is the following:

```
    COUNTERMODEL FAMILY:
- Left projection: x*y=x. Any term evaluates
  to its leftmost leaf.
- Right projection: x*y=y. Any term evaluates
  to its rightmost leaf.
- XOR on {0,1}: x*y=x+y mod 2. A law holds iff
  each variable has equal parity on both sides.
- Constant-zero: x*y=0. Useful when Eq1 collapses
  all compound terms but Eq2 exposes a variable.
- Left shift mod 3: x*y=x+1 mod 3. Tracks left spine depth.
- Right shift mod 3: x*y=y+1 mod 3. Tracks right spine depth.
- Boolean AND/OR on {0,1}. Useful for absorption/idempotence-shaped laws.
- Near-projections on {0,1,2}: start from x*y=x
  or x*y=y and perturb one row/column entry.
```